

# Fase 2 · Pipeline de obtención, exploración, limpieza, transformación y validación

MCDIA500 · Programación para la Ciencia de Datos · Grupo 6 Integrantes: *Fernanda Ovalle Román · Sebastián Cajales Cid · César Lorca Bacián*

Este notebook implementa el pipeline de la Fase 2 sobre el subconjunto definido en la Fase 1 (titulados de pregrado universitario 2025). Regla de trabajo: **medir primero, decidir después y justificar con la medición**. Cada decisión queda registrada en la bitácora (docs/bitacora.md) con sus cifras, y las funciones que la ejecutan viven en src/ con pruebas en tests/.

Antes de entregar: **Kernel → Restart Kernel and Run All Cells**. Requiere que data/processed/titulados\_2025\_pregrado\_univ.csv exista (lo genera notebooks/F1/F1\_Definicion.ipynb).

In [1]:

```
from pathlib import Path
import os, sys
RAIZ = Path.cwd()
while not (RAIZ / "README.md").exists() and RAIZ.parent != RAIZ:
    RAIZ = RAIZ.parent
os.chdir(RAIZ); sys.path.insert(0, str(RAIZ))
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from src import exploracion, limpieza, transformacion, escalado, validacion
from src.bitacora import registrar, exportar

FIG = RAIZ / "docs" / "figuras"; FIG.mkdir(parents=True, exist_ok=True)
def guardar_figura(nombre):
    ruta = FIG / f"{nombre}.png"; plt.tight_layout(); plt.savefig(ruta, dpi=120);
    plt.close(); print("figura guardada:", ruta.relative_to(RAIZ))

pd.set_option("display.width", 160); pd.set_option("display.max_columns", 30)
print("Raíz:", RAIZ, "· Python", sys.version.split()[0], "· pandas", pd.__version__)
```

Out:

```
Raíz: /tmp/claude-0/-home-claude/7d5d987b-d69f-5ff6-b6ac-07de88f524e4/scratchpad/repo ·
Python 3.11.15 · pandas 3.0.2
```

## Obtención

El dato crudo (data/raw/) no se toca. Se parte del subconjunto derivado que produce el notebook F1 y se verifica su forma antes de hacer nada.

In [2]:

```
ENTRADA = Path("data/processed/titulados_2025_pregrado_univ.csv")
assert ENTRADA.exists(), "Ejecuta primero notebooks/F1/F1_Definicion.ipynb para
```

```

| generar el subconjunto"
| df0 = pd.read_csv(ENTRADA, sep=";", encoding="utf-8", low_memory=False)
| N0 = len(df0)
| print(f"Subconjunto: {df0.shape[0]:,} filas × {df0.shape[1]} columnas")
| registrar("F2 · obtención", f"lectura de {ENTRADA} → {df0.shape[0]:,} filas ×
| {df0.shape[1]} columnas (sep ';', UTF-8)")

```

Out:

```

| Subconjunto: 105,063 filas × 40 columnas
| [F2 · obtención] lectura de data/processed/titulados_2025_pregrado_univ.csv → 105,063
| filas × 40 columnas (sep ';', UTF-8)

```

## Paso 6 · Exploración: medir antes de tocar nada

Cuatro mediciones: dimensiones y tipos, valores faltantes (incluidos los códigos de relleno), categorías observadas y valores atípicos.

In [3]:

```

| # 1) Dimensiones y tipos
| print(df0.dtypes.value_counts().rename("columnas").to_string())
| print()
| # 2) Faltantes reales + códigos de relleno (1900 en año de ingreso, 0 en semestre de
| ingreso)
| perfil = exploracion.perfil_faltantes(df0, codigos={"anio_ing_carr_ori": 1900,
| "sem_ing_carr_ori": 0, "cod_sede": 0})
| print(perfil[perfil["pct_total"] > 0].to_string())

```

Out:

```

| str      24
| int64    13
| float64   3

```

|                   | nulos | pct_nulos | codigo_relleno | pct_codigo | pct_total |
|-------------------|-------|-----------|----------------|------------|-----------|
| columna           |       |           |                |            |           |
| sem_ing_carr_ori  | 0     | 0.00      | 13396          | 12.75      | 12.75     |
| anio_ing_carr_ori | 0     | 0.00      | 13396          | 12.75      | 12.75     |
| nombre_grado      | 8437  | 8.03      | 0              | 0.00       | 8.03      |
| nombre_titulo     | 2997  | 2.85      | 0              | 0.00       | 2.85      |
| cod_carrera       | 300   | 0.29      | 0              | 0.00       | 0.29      |
| version           | 290   | 0.28      | 0              | 0.00       | 0.28      |
| mrnun             | 7     | 0.01      | 0              | 0.00       | 0.01      |

In [4]:

```

| # ¿Los registros con código 1900 tienen duraciones distintas? (decide si el código
| es informativo o solo faltante)
| m1900 = df0["anio_ing_carr_ori"] == 1900
| comp = pd.DataFrame({"con código 1900": df0.loc[m1900, ["dur_estudio_carr",
| "dur_total_carr"]].describe().loc[["count", "mean", "50%", "max"]].round(2).stack(),
| "sin código": df0.loc[~m1900, ["dur_estudio_carr",
| "dur_total_carr"]].describe().loc[["count", "mean", "50%",
| "max"]].round(2).stack()})
| print(comp.to_string())
| print()
| print("anio_ing_carr_ori == anio_ing_carr_act en registros válidos:",

```

```
f"{df0.loc[~m1900, 'anio_ing_carr_ori'] == df0.loc[~m1900, 'anio_ing_carr_act']).mean():.1%}")
```

Out:

|       |                  | con código 1900 | sin código |
|-------|------------------|-----------------|------------|
| count | dur_estudio_carr | 13396.00        | 91667.00   |
|       | dur_total_carr   | 13396.00        | 91667.00   |
| mean  | dur_estudio_carr | 5.55            | 9.54       |
|       | dur_total_carr   | 5.55            | 9.59       |
| 50%   | dur_estudio_carr | 6.00            | 10.00      |
|       | dur_total_carr   | 6.00            | 10.00      |
| max   | dur_estudio_carr | 14.00           | 24.00      |
|       | dur_total_carr   | 14.00           | 24.00      |

anio\_ing\_carr\_ori == anio\_ing\_carr\_act en registros válidos: 91.2%

In [5]:

```
# 3) Categorías observadas: detecta variantes de escritura y niveles raros
cat = exploracion.resumen_categorias(df0, ["tipo_inst_2", "jornada", "modalidad",
"rango_edad", "nivel_carrera_1", "tipo_plan_carr", "area_conocimiento"], top=6)
for c, r in cat.iterrows():
    print(f"{c} ({r['n_categorias']}): {r['mas_frecuentes']}")
print()
print("nomb_carrera:", df0["nomb_carrera"].nunique(), "valores;",
(df0["nomb_carrera"].value_counts() < 100).sum(), "con menos de 100 registros")
print("nomb_inst : ", df0["nomb_inst"].nunique(), "valores;",
(df0["nomb_inst"].value_counts() < 100).sum(), "con menos de 100 registros")
```

Out:

```
tipo_inst_2 (2): Universidades CRUCH (55,572), Universidades Privadas (49,491)
jornada (5): Diurna (82,197), Vespertina (12,431), A Distancia (9,064), Semipresencial
(828), Otra (543)
modalidad (3): Presencial (94,874), No Presencial (9,064), Semipresencial (1,125)
rango_edad (6): 20 a 24 Años (46,399), 25 a 29 Años (36,078), 30 a 34 Años (9,798), 40
y más años (6,868), 35 a 39 Años (5,759), 15 a 19 Años (161)
nivel_carrera_1 (4): Profesional Con Licenciatura (93,629), Técnico de Nivel Superior
(5,329), Profesional Sin Licenciatura (3,108), Licenciatura No Conducente a Título
(2,997)
tipo_plan_carr (3): Plan Regular (86,937), Plan Regular de Continuidad (15,472), Plan
Especial (2,654)
area_conocimiento (10): Tecnología (23,705), Salud (22,506), Ciencias Sociales
(16,371), Administración y Comercio (14,295), Educación (11,833), Derecho (5,074)

nomb_carrera: 1096 valores; 968 con menos de 100 registros
nomb_inst : 55 valores; 3 con menos de 100 registros
```

In [6]:

```
# 4) Valores atípicos por rango intercuartílico
at = pd.DataFrame({c: exploracion.atipicos_ric(df0[c]) for c in ["dur_estudio_carr",
"dur_proceso_tit", "dur_total_carr"]}).T
print(at.to_string())
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2))
for ax, c in zip(axes, ["dur_estudio_carr", "dur_proceso_tit", "dur_total_carr"]):
    ax.boxplot(df0[c].dropna(), vert=True); ax.set_title(c, fontsize=9);
    ax.set_ylabel("semestres", fontsize=8)
```

```

guardar_figura("f2_01_atipicos_duraciones")

registrar("F2 · exploración", f"código 1900 en anio_ing_carr_ori:
{int(m1900.sum()):,} registros (12,8 %) con duración mediana "

        f"{df0.loc[m1900,'dur_total_carr'].median():.0f} vs
{df0.loc[~m1900,'dur_total_carr'].median():.0f} semestres → no es un faltante
aleatorio")

registrar("F2 · exploración", f"dur_total_carr: mediana
{df0.dur_total_carr.median():.0f} semestres, RIC
{at.loc['dur_total_carr','ric']:.0f}, "

        f"{int(at.loc['dur_total_carr','n_atipicos']):,} atípicos
({at.loc['dur_total_carr','pct']:.1f} %) por regla 1,5·RIC → hay valores extremos")

```

Out:

```

      q1    q3   ric  lim_inf  lim_sup  n_atipicos   pct
dur_estudio_carr  8.0  10.0  2.0     5.0    13.0    10162.0  9.67
dur_proceso_tit   0.0   1.0  1.0    -1.5     2.5     312.0  0.30
dur_total_carr    8.0  10.0  2.0     5.0    13.0     9845.0  9.37

figura guardada: docs/figuras/f2_01_atipicos_duraciones.png

[F2 · exploración] código 1900 en anio_ing_carr_ori: 13,396 registros (12,8 %) con
duración mediana 6 vs 10 semestres → no es un faltante aleatorio

[F2 · exploración] dur_total_carr: mediana 10 semestres, RIC 2, 9,845 atípicos (9.4 %)
por regla 1,5·RIC → hay valores extremos

```

In [7]:

```

# Vista previa de la pregunta: mediana de duración por factor (solo descriptivo; el
análisis es Fase 3-4)

for f in ["tipo_inst_2", "jornada", "modalidad"]:

    print(df0.groupby(f)["dur_total_carr"].agg(n="size", mediana="median",
media="mean").round(2).sort_values("mediana").to_string()); print()

```

Out:

```

      n  mediana  media
tipo_inst_2
Universidades CRUCH    55572    10.0   9.50
Universidades Privadas  49491    10.0   8.59

```

```

      n  mediana  media
jornada
A Distancia    9064     6.0   5.37
Vespertina    12431     6.0   6.41
Semipresencial  828     7.0   6.70
Otra           543     8.0   6.97
Diurna        82197    10.0   9.92

```

```

      n  mediana  media
modalidad
No Presencial    9064     6.0   5.37
Semipresencial  1125     6.0   6.68
Presencial      94874    10.0   9.46

```

## Paso 7 · Limpieza e imputación: comparar antes de decidir

Decisiones, en orden: duplicados → columnas constantes → códigos de relleno como faltantes

→ estrategia de imputación elegida por su efecto sobre la dispersión.

In [8]:

```
df1, n_dup = limpieza.eliminar_duplicados(df0)

registrar("F2 · limpieza", f"duplicados exactos: {n_dup} filas eliminadas
({100*n_dup/N0:.3f} %) → {len(df1):,} filas")

df1, constantes = limpieza.eliminar_columnas_constantes(df1)

registrar("F2 · limpieza", f"columnas constantes eliminadas (un solo valor tras el
filtro de F1): {constantes}")

df1, n1900 = limpieza.codigo_a_nulo(df1, "anio_ing_carr_ori", 1900)
df1, n0 = limpieza.codigo_a_nulo(df1, "sem_ing_carr_ori", 0)

registrar("F2 · limpieza", f"anio_ing_carr_ori: código 1900 tratado como faltante en
{n1900:,} filas ({100*n1900/len(df1):.1f} %); "
        f"sem_ing_carr_ori: código 0 tratado como faltante en {n0:,} filas
({100*n0/len(df1):.1f} %)")
```

Out:

```
[F2 · limpieza] duplicados exactos: 3 filas eliminadas (0.003 %) → 105,060 filas
[F2 · limpieza] columnas constantes eliminadas (un solo valor tras el filtro de F1):
['cat_periodo', 'tipo_inst_1', 'nivel_global']
[F2 · limpieza] anio_ing_carr_ori: código 1900 tratado como faltante en 13,396 filas
(12.8 %); sem_ing_carr_ori: código 0 tratado como faltante en 13,396 filas (12.8 %)
```

In [9]:

```
# Comparación de estrategias para anio_ing_carr_ori: eliminar, media, mediana,
mediana por grupo (anio_ing_carr_act)

comparacion = limpieza.comparar_imputaciones(df1, "anio_ing_carr_ori",
grupo="anio_ing_carr_act")

print(comparacion.to_string())
```

Out:

|                                       | n      | media    | desv  | cambio_desv_pct |
|---------------------------------------|--------|----------|-------|-----------------|
| estrategia                            |        |          |       |                 |
| observados (referencia)               | 91664  | 2019.449 | 2.561 | 0.000           |
| eliminar filas                        | 91664  | 2019.449 | 2.561 | 0.000           |
| imputar media                         | 105060 | 2019.449 | 2.392 | -6.593          |
| imputar mediana                       | 105060 | 2019.519 | 2.399 | -6.318          |
| imputar mediana por anio_ing_carr_act | 105060 | 2019.812 | 2.633 | 2.821           |

**Lectura de la tabla.** Los registros con código 1900 **no son aleatorios**: su duración mediana es de 6 semestres frente a 10 en el resto (la exploración lo mostró), lo que sugiere estudiantes que cambiaron de carrera y cuyo ingreso original no quedó registrado. Eliminarlos descartaría casi el 13 % de la muestra y sesgaría la duración hacia arriba. La media y la mediana global llevan a todos los faltantes al mismo año y reducen la dispersión en más de 6 %. La mediana **por año de ingreso a la carrera actual** usa una variable observada que coincide con el año de origen en el 91 % de los registros válidos y es la que menos deforma la distribución (cambio de la desviación de +2,8 % frente a -6,3 % y -6,6 %). Es la estrategia elegida; la bandera anio\_ing\_carr\_ori\_imputada permite excluir esos registros en un análisis de sensibilidad.

In [10]:

```
df2, cifras = limpieza.imputar_mediana(df1, "anio_ing_carr_ori",
grupo="anio_ing_carr_act")

registrar("F2 · limpieza", f"anio_ing_carr_ori: {cifras['n_imputados'],} nulos
({cifras['pct']} %) imputados con {cifras['valor']}"; "
        f"cambio en la desviación estándar {cifras['cambio_desv_pct']:+.2f} %;
bandera anio_ing_carr_ori_imputada")
```

```
df2, cifras_s = limpieza.imputar_mediana(df2, "sem_ing_carr_ori")

registrar("F2 · limpieza", f"sem_ing_carr_ori: {cifras_s['n_imputados'],} nulos
({cifras_s['pct']} %) imputados con la mediana = {cifras_s['valor']:.0f}")

# Nulos residuales en columnas que NO son de análisis: se conservan como texto
(nombre_titulo / nombre_grado son descriptivos)

print(df2[["nombre_titulo", "nombre_grado", "cod_carrera", "version",
"mrn"]].isna().sum().to_string())

registrar("F2 · limpieza", "nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se
conservan sin imputar: son etiquetas de texto, no variables de análisis; "

"mrn con 7 nulos se conserva porque el análisis no cuenta personas sino
títulos")
```

Out:

```
[F2 · limpieza] anio_ing_carr_ori: 13,396 nulos (12.75 %) imputados con mediana por
anio_ing_carr_act; cambio en la desviación estándar +2.82 %; bandera
anio_ing_carr_ori_imputada

[F2 · limpieza] sem_ing_carr_ori: 13,396 nulos (12.75 %) imputados con la mediana = 1

nombre_titulo      2997
nombre_grado       8434
cod_carrera         297
version            287
mrn                 7

[F2 · limpieza] nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se conservan sin
imputar: son etiquetas de texto, no variables de análisis; mrn con 7 nulos se conserva
porque el análisis no cuenta personas sino títulos
```

## Paso 8 · Transformación por tipo de variable

Cada rol exige un tratamiento distinto: ordinales con **orden declarado por el equipo**, nominales con one-hot, alta cardinalidad agrupada, fechas parseadas y derivadas.

In [11]:

```
df3, mapa_edad = transformacion.codificar_ordinal(df2, "rango_edad",
transformacion.ORDEN_RANGO_EDAD)

df3, mapa_nivel = transformacion.codificar_ordinal(df3, "nivel_carrera_1",
transformacion.ORDEN_NIVEL_CARRERA)

print("rango_edad →", mapa_edad); print("nivel_carrera_1 →", mapa_nivel)

fuera = df3.attrs.get("rango_edad_fuera_de_orden", [])

n_fuera = int(df3["rango_edad_ord"].isna().sum())

registrar("F2 · transformación", f"ordinales codificadas con orden declarado:
rango_edad (6 niveles) y nivel_carrera_1 (4 niveles); "

f"{n_fuera} registros con rango_edad {fuera} quedan como faltante")
```

Out:

```
rango_edad → {'15 a 19 Años': 0, '20 a 24 Años': 1, '25 a 29 Años': 2, '30 a 34 Años':
3, '35 a 39 Años': 4, '40 y más años': 5}

nivel_carrera_1 → {'Técnico de Nivel Superior': 0, 'Licenciatura No Conducente a
Título': 1, 'Profesional Sin Licenciatura': 2, 'Profesional Con Licenciatura': 3}

[F2 · transformación] ordinales codificadas con orden declarado: rango_edad (6 niveles)
y nivel_carrera_1 (4 niveles); 0 registros con rango_edad [] quedan como faltante
```

In [12]:

```
# Si hubiera 'Sin Información' en rango_edad, se recupera desde la edad calculada
con las fechas (derivación), no se inventa

df3 = transformacion.derivar_fechas(df3)

print(df3[["edad_titulacion", "mes_titulacion"]].describe().round(1).to_string())
```

```

sin_info = df3["rango_edad_ord"].isna()
bins = [0, 20, 25, 30, 35, 40, 200]
df3.loc[sin_info, "rango_edad_ord"] = pd.cut(df3.loc[sin_info, "edad_titulacion"],
bins=bins, right=False, labels=False)
df3["rango_edad_ord"] = df3["rango_edad_ord"].astype(int)
registrar("F2 · transformación", f"fechas: fecha_obtencion_titulo y fec_nac_alu
parseadas; derivadas mes_titulacion y edad_titulacion "
f"(mediana {df3.edad_titulacion.median():.1f} años); {int(sin_info.sum())}
'Sin Información' de rango_edad recuperados desde edad_titulacion")

```

Out:

|       | edad_titulacion | mes_titulacion |
|-------|-----------------|----------------|
| count | 105060.0        | 105060.0       |
| mean  | 28.0            | 6.4            |
| std   | 6.4             | 3.9            |
| min   | 19.2            | 1.0            |
| 25%   | 24.1            | 3.0            |
| 50%   | 25.6            | 6.0            |
| 75%   | 29.1            | 10.0           |
| max   | 80.9            | 12.0           |

[F2 · transformación] fechas: fecha\_obtencion\_titulo y fec\_nac\_alu parseadas; derivadas mes\_titulacion y edad\_titulacion (mediana 25.6 años); 0 'Sin Información' de rango\_edad recuperados desde edad\_titulacion

In [13]:

```

NOMINALES = ["tipo_inst_2", "jornada", "modalidad", "area_conocimiento",
"region_sede"]
df4, cols_onehot = transformacion.one_hot(df3, NOMINALES)
grupos = {c: [k for k in cols_onehot if k.startswith(c + "_")] for c in NOMINALES}
print({c: len(v) for c, v in grupos.items()})
registrar("F2 · transformación", f"one-hot encoding de {len(NOMINALES)} nominales →
{len(cols_onehot)} columnas 0/1")

df4, n_raras = transformacion.agrupar_raras(df4, "nomb_carrera", min_frec=100)
registrar("F2 · transformación", f"nomb_carrera: {n_raras:,} categorías con < 100
registros agrupadas en 'OTRA' → "
f"{df4['nomb_carrera_agrupada'].nunique()} categorías")

df4 = transformacion.binaria_01(df4, "gen_alu", 2, "mujer")
registrar("F2 · transformación", f"gen_alu recodificada como mujer (0/1):
{df4.mujer.mean():.1%} mujeres")

```

Out:

```

{'tipo_inst_2': 2, 'jornada': 5, 'modalidad': 3, 'area_conocimiento': 10,
'region_sede': 16}
[F2 · transformación] one-hot encoding de 5 nominales → 36 columnas 0/1
[F2 · transformación] nomb_carrera: 968 categorías con < 100 registros agrupadas en
'OTRA' → 129 categorías
[F2 · transformación] gen_alu recodificada como mujer (0/1): 56.3% mujeres

```

## Paso 9 · Escalamiento: comparar los tres

La exploración mostró valores extremos ( $\approx 9\%$  de atípicos en `dur_total_carr`) y la limpieza imputó con mediana. Por coherencia, el escalador debe centrar en la mediana, no en la media.

In [14]:

```
comp_esc = escalado.comparar_escaladores(df4["dur_total_carr"])
print(comp_esc.to_string())
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2))
for ax, (nombre, f) in zip(axes, [("estandar", escalado.escalar_estandar),
("minmax", escalado.escalar_minmax), ("robusto", escalado.escalar_robusto)]):
    ax.hist(f(df4["dur_total_carr"]), bins=40); ax.set_title(f"dur_total_carr ·
{nombre}", fontsize=9)
guardar_figura("f2_02_escaladores")
```

Out:

|          | media  | desv  | mediana | ric   | min    | max    |
|----------|--------|-------|---------|-------|--------|--------|
| original | 9.074  | 2.440 | 10.000  | 2.000 | 1.000  | 24.000 |
| estandar | -0.000 | 1.000 | 0.380   | 0.820 | -3.309 | 6.116  |
| minmax   | 0.351  | 0.106 | 0.391   | 0.087 | 0.000  | 1.000  |
| robusto  | -0.463 | 1.220 | 0.000   | 1.000 | -4.500 | 7.000  |

figura guardada: docs/figuras/f2\_02\_escaladores.png

In [15]:

```
ESCALAR = ["dur_total_carr", "dur_estudio_carr", "edad_titulacion"]
for c in ESCALAR:
    df4[f"{c}_esc"] = escalado.escalar_robusto(df4[c])
registrar("F2 · escalamiento", f"RobustScaler (mediana 0, RIC 1) aplicado a
{ESCALAR}: coherente con la imputación por mediana y con los "
        f"atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango
estrecho por el máximo de 24 semestres")
```

Out:

[F2 · escalamiento] RobustScaler (mediana 0, RIC 1) aplicado a ['dur\_total\_carr', 'dur\_estudio\_carr', 'edad\_titulacion']: coherente con la imputación por mediana y con los atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho por el máximo de 24 semestres

## Paso 10 · Validación: demostrar, no afirmar

Aserciones sobre el dataset final, comprobación de la propiedad de cada escalador y pruebas de las funciones (caso normal, límite y excepción).

In [16]:

```
COLS_ANALISIS = ["dur_total_carr", "dur_estudio_carr", "dur_proceso_tit",
"edad_titulacion", "mujer",
                "rango_edad_ord", "nivel_carrera_1_ord", "anio_ing_carr_ori",
"anio_ing_carr_ori_imputada"] + cols_onehot + [f"{c}_esc" for c in ESCALAR]
checks = validacion.validar_dataset_final(df4, COLS_ANALISIS, grupos,
n_filas_esperado=N0 - n_dup)
for c in checks: print("OK", c)
for c in ESCALAR:
    assert validacion.verificar_propiedad_escalador(df4[f"{c}_esc"], "robusto"), c
print("OK propiedad del escalador robusto verificada en", ESCALAR)
registrar("F2 · validación", f"{len(checks)} comprobaciones superadas sobre
{len(COLS_ANALISIS)} columnas de análisis; propiedad mediana 0 / RIC 1 verificada")
```

Out:

OK sin nulos en columnas de análisis  
OK todas las columnas de análisis son numéricas  
OK cada grupo one-hot suma 1 por fila (5 grupos)  
OK sin filas duplicadas

```
OK se conservó el número de filas esperado (105,060)
```

```
OK propiedad del escalador robusto verificada en ['dur_total_carr', 'dur_estudio_carr', 'edad_titulacion']
```

```
[F2 · validación] 5 comprobaciones superadas sobre 48 columnas de análisis; propiedad mediana 0 / RIC 1 verificada
```

```
In [17]:
```

```
import subprocess

res = subprocess.run([sys.executable, "tests/test_pipeline.py"],
capture_output=True, text=True)

print(res.stdout[-1500:]); assert res.returncode == 0, res.stderr

registrar("F2 · validación", "pruebas unitarias de src/ (tests/test_pipeline.py): "
+ res.stdout.strip().splitlines()[-1])
```

```
Out:
```

```
OK      test_atipicos_ric_detecta_extremo
OK      test_atipicos_serie_vacia
OK      test_codigo_a_nulo_normal
OK      test_columna_sin_nulos_no_imputa_nada
OK      test_eliminar_duplicados_sin_duplicados
OK      test_escaladores_cumplen_propiedad
OK      test_excepciones
OK      test_imputar_mediana_crea_bandera
OK      test_one_hot_suma_uno
OK      test_ordinal_respeto_orden_declarado
OK      test_una_sola_categoria_one_hot
```

```
11/11 pruebas superadas
```

```
[F2 · validación] pruebas unitarias de src/ (tests/test_pipeline.py): 11/11 pruebas superadas
```

## Paso 11 · Persistencia y bitácora

El resultado va a data/processed/, nunca sobre data/raw/. Se relea para comprobar forma y columnas, y se exporta la bitácora.

```
In [18]:
```

```
SALIDA = Path("data/processed/titulados_2025_pregrado_univ_limpio.csv")
df4.to_csv(SALIDA, sep=";", index=False, encoding="utf-8")
chk = pd.read_csv(SALIDA, sep=";", low_memory=False)
assert chk.shape == df4.shape and list(chk.columns) == list(df4.columns)

print(f"Guardado y verificado: {SALIDA} → {chk.shape[0]:,} filas × {chk.shape[1]}
columnas · {SALIDA.stat().st_size/1_048_576:.1f} MB")

registrar("F2 · persistencia", f"{SALIDA.name}: {chk.shape[0]:,} × {chk.shape[1]},
releído y verificado (forma y columnas)")

exportar("docs/bitacora.md")

print(Path("docs/bitacora.md").read_text(encoding="utf-8"))
```

```
Out:
```

```
Guardado y verificado: data/processed/titulados_2025_pregrado_univ_limpio.csv → 105,060
filas × 84 columnas · 66.7 MB
```

```
[F2 · persistencia] titulados_2025_pregrado_univ_limpio.csv: 105,060 × 84, releído y
verificado (forma y columnas)
```

```
| Bitácora exportada a docs/bitacora.md (18 decisiones)
| # Bitácora de decisiones

| Generada el 2026-09-12 20:03

| - [F2 · obtención] lectura de data/processed/titulados_2025_pregrado_univ.csv → 105,063
|   filas × 40 columnas (sep ';', UTF-8)
|
| - [F2 · exploración] código 1900 en anio_ing_carr_ori: 13,396 registros (12,8 %) con
|   duración mediana 6 vs 10 semestres → no es un faltante aleatorio
|
| - [F2 · exploración] dur_total_carr: mediana 10 semestres, RIC 2, 9,845 atípicos (9.4
|   %) por regla 1,5·RIC → hay valores extremos
|
| - [F2 · limpieza] duplicados exactos: 3 filas eliminadas (0.003 %) → 105,060 filas
|
| - [F2 · limpieza] columnas constantes eliminadas (un solo valor tras el filtro de F1):
|   ['cat_periodo', 'tipo_inst_1', 'nivel_global']
|
| - [F2 · limpieza] anio_ing_carr_ori: código 1900 tratado como faltante en 13,396 filas
|   (12.8 %); sem_ing_carr_ori: código 0 tratado como faltante en 13,396 filas (12.8 %)
|
| - [F2 · limpieza] anio_ing_carr_ori: 13,396 nulos (12.75 %) imputados con mediana por
|   anio_ing_carr_act; cambio en la desviación estándar +2.82 %; bandera
|   anio_ing_carr_ori_imputada
|
| - [F2 · limpieza] sem_ing_carr_ori: 13,396 nulos (12.75 %) imputados con la mediana = 1
|
| - [F2 · limpieza] nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se conservan sin
|   imputar: son etiquetas de texto, no variables de análisis; mrun con 7 nulos se conserva
|   porque el análisis no cuenta personas sino títulos
|
| - [F2 · transformación] ordinales codificadas con orden declarado: rango_edad (6
|   niveles) y nivel_carrera_1 (4 niveles); 0 registros con rango_edad [] quedan como
|   faltante
|
| - [F2 · transformación] fechas: fecha_obtencion_titulo y fec_nac_alu parseadas;
|   derivadas mes_titulacion y edad_titulacion (mediana 25.6 años); 0 'Sin Información' de
|   rango_edad recuperados desde edad_titulacion
|
| - [F2 · transformación] one-hot encoding de 5 nominales → 36 columnas 0/1
|
| - [F2 · transformación] nomb_carrera: 968 categorías con < 100 registros agrupadas en
|   'OTRA' → 129 categorías
|
| - [F2 · transformación] gen_alu recodificada como mujer (0/1): 56.3% mujeres
|
| - [F2 · escalamiento] RobustScaler (mediana 0, RIC 1) aplicado a ['dur_total_carr',
|   'dur_estudio_carr', 'edad_titulacion']: coherente con la imputación por mediana y con
|   los atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho
|   por el máximo de 24 semestres
|
| - [F2 · validación] 5 comprobaciones superadas sobre 48 columnas de análisis; propiedad
|   mediana 0 / RIC 1 verificada
|
| - [F2 · validación] pruebas unitarias de src/ (tests/test_pipeline.py): 11/11 pruebas
|   superadas
|
| - [F2 · persistencia] titulados_2025_pregrado_univ_limpio.csv: 105,060 × 84, releído y
|   verificado (forma y columnas)
```

Resumen del pipeline

| Paso        | Entrada → salida                          | Decisión clave                                           |
|-------------|-------------------------------------------|----------------------------------------------------------|
| Obtención   | subconjunto F1 (105.063 × 40)             | dato crudo intacto                                       |
| Exploración | perfil de faltantes, categorías, atípicos | código 1900 no es un año: es faltante                    |
| Limpieza    | −3 duplicados, −3                         | mediana por grupo (menor deformación de la dispersión) + |

| Paso           | Entrada → salida                                         | Decisión clave                                                             |
|----------------|----------------------------------------------------------|----------------------------------------------------------------------------|
|                | constantes, 1900/0 → NaN                                 | bandera                                                                    |
| Transformación | ordinales, one-hot, agrupación, fechas                   | orden declarado explícitamente; 'Sin Información' recuperado desde la edad |
| Escalamiento   | 3 duraciones/edad escaladas                              | RobustScaler, coherente con mediana y atípicos                             |
| Validación     | 5 asserts + 11 pruebas                                   | todo verificado, no afirmado                                               |
| Persistencia   | data/<br>processed/..._limpio.<br>csv + docs/bitacora.md | relectura comprobada                                                       |